

实验二 七段数码管动态显示电路设计报告

姓名：怀迪 学号：PB23061112

一、实验内容简述

本次实验大体上是要求我们编写 Verilog 代码实现一个七段数码管动态显示电路设计，并在 Quartus 软件中进行仿真和 FPGA 验证。

需要实现的具体功能: 根据输入的位选信号（3 位）选择需要改变输出内容的数码管；然后输入数据信号（4 位），来控制输出的 8 个阳极共通的数码管（每一个阳极对应一整个七段数码管）的 7 个阴极（对应每一个数码管显示的数字）；最后发送使能信号，改变数码管上的数字显示。

二、设计分析

1. 实验题目分析

首先我们需要确认输入输出信号一共需要 8 位：3 位位选信号 + 4 位数据信号 + 1 位使能信号 + 1 个 clk 信号。

3 位位选信号需要用来选择 8 个数码管中的一个进行改变数值，4 位数据信号用来给不同的数码管进行赋值，1 位使能信号用来控制数码管的显示改变与否，clk 信号用来循环显示数码管。

而输出需要控制 8 个数码管的 8 个阴极信号 + 控制数码管 LED 灯显示的 8 个信号，一共十六位。

明确了输入信号输出信号的数量我们就需要思考如何进行功能实现：首先我们需要一个计数器来对 clk 信号进行分频（分频前 clk 为 50Mhz，过大），然后我们需要根据位选信号选择寄存器来存储数码管的数值，之后我们需要一个循环计数器来循环选择数码管显示其对应寄存器存储的数值，循环显示的频率为分频后的信号频率，最后根据寄存器的数值来解码成 LED 灯的阴极信号以实现数字显示。

2. 编程思路

我把代码分成了三个模块: 实现具体功能的顶层模块, 分频模块和解码信号模块。

2.1 顶层模块

顶层模块主要实现对输入信号的处理和对其他两个模块的调用。首先调用分频模块对 `clk` 信号进行分频, 得到一个较低频率的时钟信号 `clk_divided`。

然后根据想要改变数值的数码管选择寄存器进行赋值, 使用一个 `case` 语句根据 3 位选信号选择对应的寄存器进行赋值操作。

接着使用一个循环计数器根据分频后的时钟信号循环选择数码管进行显示, 使用一个 `case` 语句根据循环计数器的值选择对应的数码管进行显示。

最后调用解码模块将寄存器的数值解码成对应的阴极信号。

2.2 分频模块

分频模块主要实现对输入的 `clk` 信号进行分频操作。

首先输入未分频的时钟信号, 使用一个计数器对输入的 `clk` 信号进行计数, 当计数器达到设定的分频值时, 翻转输出的分频信号, 并将计数器清零。

此时得到的输出信号为分频后的时钟信号 `clk_divided`。

2.3 解码模块

解码模块主要实现将 4 位二进制数值解码成对应的七段数码管阴极信号。

首先输入 4 位二进制数值, 使用一个 `case` 语句根据输入的数值选择对应的七段数码管阴极信号。

输出对应的七段数码管阴极信号以实现数字显示。

三、VHDL 源代码

本代码实现了分频时钟作为输入, 选择相应寄存器并手动输入数据实现更改数码管显示内容的功能, 同时实现循环选择数码管进行动态显示的功能。其中, 每次更改数码管显示内容的时候, 需要将使能信号拉低, 完成更改后再拉高。我还添加了复位功能, 当复位信号拉低时, 所有寄存器置位, 数码管显示 1-8。

```

1 timescale 10ns/1ns
2 module FPGA_EXP2_hd(
3     input rst,
4     input clk,
5     input ena,
6     input [3:0] input_data,
7     input [2:0] reg_choose,
8     output DP,
9     output [6:0] LED_data, //reg[6]->a,reg[0]->g
10    output reg[7:0] LED_choose,
11    output clkout_debug
12 );
13
14 //1.时钟分频
15 wire clk_divided; //分频后的时钟信号
16 div50MHz div1(
17     .clk(clk),
18     .rst(rst),
19     .clkout(clk_divided)
20 );
21 assign clkout_debug = clk_divided;
22
23 //2.数据锁存
24 reg [3:0] data_S0; //记录输入数据
25 reg [3:0] data_S1;
26 reg [3:0] data_S2;
27 reg [3:0] data_S3;
28 reg [3:0] data_S4;
29 reg [3:0] data_S5;
30 reg [3:0] data_S6;
31 reg [3:0] data_S7;
32 assign DP=1'b0; //小数点常亮
33
34 always @(posedge ena or negedge rst) //赋值
35 begin
36     if(!rst) //复位信号
37     begin
38         data_S0<=4'b0001;
39         data_S1<=4'b0010;
40         data_S2<=4'b0011;
41         data_S3<=4'b0100;
42         data_S4<=4'b0101;
43         data_S5<=4'b0110;
44         data_S6<=4'b0111;
45         data_S7<=4'b1000;
46     end
47
48     else if(ena)
49     begin
50         case (reg_choose)
51             3'b000: data_S0<=input_data;
52             3'b001: data_S1<=input_data;
53             3'b010: data_S2<=input_data;
54             3'b011: data_S3<=input_data;
55             3'b100: data_S4<=input_data;
56             3'b101: data_S5<=input_data;
57             3'b110: data_S6<=input_data;
58             3'b111: data_S7<=input_data;
59         endcase
60     end
61
62 //3.位选控制,循环选中LED灯,频率为1000hz
63 reg [3:0] output_data; //输出给LED灯的数据
64 reg [2:0] num; //用来循环给LED灯赋值
65 always @(posedge clk_divided) //循环给LED灯赋值
66 begin
67     if(num == 3'b111)
68     begin
69         num <= 3'b000;
70     end
71     else if(num>=3'b000 && num<=3'b110)
72     begin
73         num <= num + 3'b001;
74     end
75     else
76     begin
77         num <= 3'b000;
78     end
79
80     case (num)
81         3'b000: begin output_data<=data_S0; LED_choose<=8'b00000001; end
82         3'b001: begin output_data<=data_S1; LED_choose<=8'b00000010; end
83         3'b010: begin output_data<=data_S2; LED_choose<=8'b00000100; end
84         3'b011: begin output_data<=data_S3; LED_choose<=8'b00001000; end
85         3'b100: begin output_data<=data_S4; LED_choose<=8'b00010000; end
86         3'b101: begin output_data<=data_S5; LED_choose<=8'b00100000; end
87         3'b110: begin output_data<=data_S6; LED_choose<=8'b01000000; end
88         3'b111: begin output_data<=data_S7; LED_choose<=8'b10000000; end
89     endcase
90 end
91
92

```

图 1: 设计代码 1-92 行图片

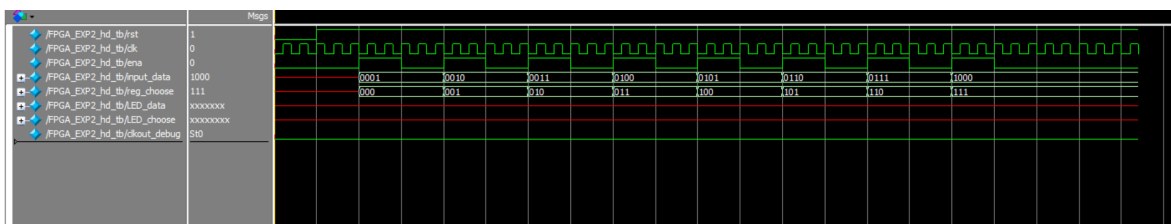
```

93 //4. 段码译码,4位数据--共阳极8段信号
94 module LED_decode decode1(
95     .in(output_data),
96     .out(LED_data)
97 );
98
99 endmodule
100
101 module div50MHz //分频模块
102 input clk; // 输入: wire类型 (50MHz)
103 input rst;
104 output reg clkout; // 输出: reg类型 (分频后的时钟, 1kHz)
105
106 // 分频 (通过计数器实现 50MHz-1kHz)
107 reg [15:0] cnt; // 计数器
108 always @(posedge clk)
109 begin
110     if (cnt >= 5'b11111)
111     begin
112         cnt <= 1'b0;
113         clkout <= ~clkout; // 翻转输出, 生成 1kHz 方波
114     end
115     else
116     begin
117         cnt <= cnt + 1'b1;
118     end
119 end
120 endmodule
121
122 module LED_decode //将值赋给数码管的PIN
123 input [3:0] in,
124 output reg [6:0] out
125
126 always @(*)
127 begin
128     case (in)
129         4'b0000: out = 7'b0000001; //0
130         4'b0001: out = 7'b0001111; //1
131         4'b0010: out = 7'b0010010; //2
132         4'b0011: out = 7'b0000110; //3
133         4'b0100: out = 7'b1001100; //4
134         4'b0101: out = 7'b0100100; //5
135         4'b0110: out = 7'b0100000; //6
136         4'b0111: out = 7'b0001111; //7
137         4'b1000: out = 7'b0000000; //8
138         4'b1001: out = 7'b0000100; //9
139         4'b1010: out = 7'b0001000; //A
140         4'b1011: out = 7'b1100000; //B
141         4'b1100: out = 7'b0110001; //C
142         4'b1101: out = 7'b1000010; //D
143         4'b1110: out = 7'b1000000; //E
144         4'b1111: out = 7'b0111000; //F
145     endcase
146 end
147 endmodule
148

```

图 2: 设计代码 93-149 行图片

四、仿真结果记录



仿真结果如上图所示, 首先复位信号拉低, 所有寄存器置位, 数码管显示 1-8;

然后复位信号拉高，顺序选择数码管更改数值，分别将数码管 1-8 更改为 1、2、3、4、5、6、7、8，每次更改数值时使能信号拉低，完成更改后使能信号拉高。

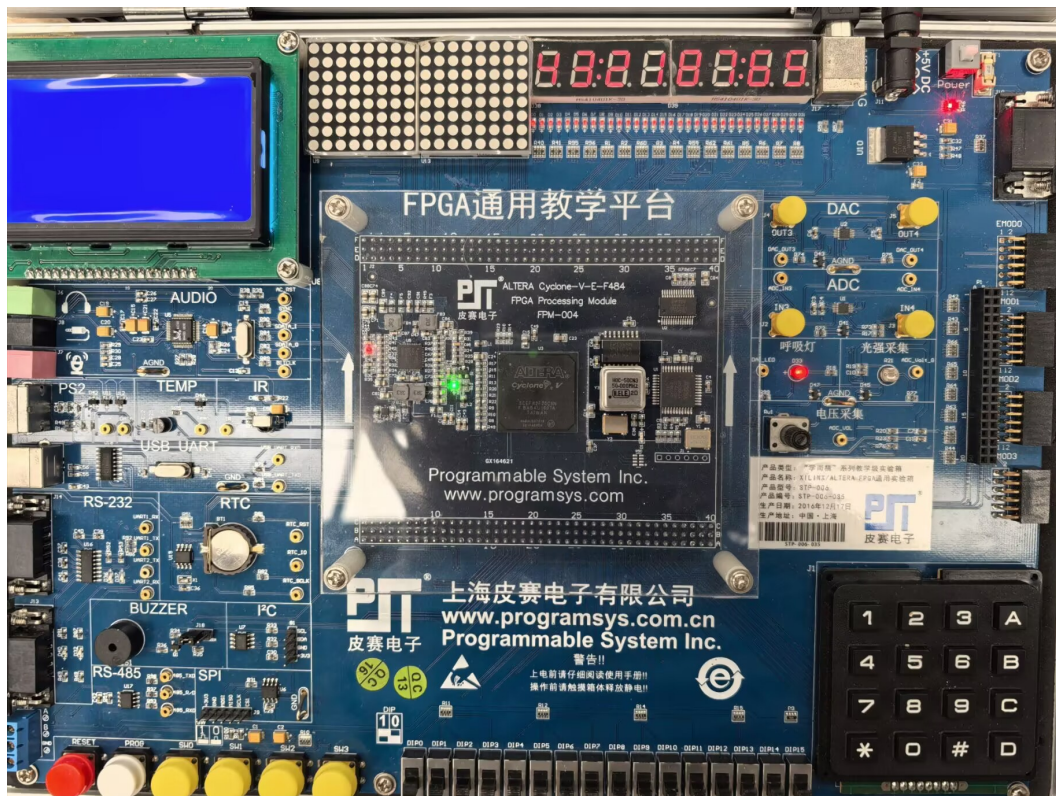
可能是软件原因，仿真波形图中数码管显示的内容没有体现出来，但实际上代码逻辑是正确的，FPGA 验证结果也符合预期。后续有待进一步排查。

五、FPGA 验证结果记录

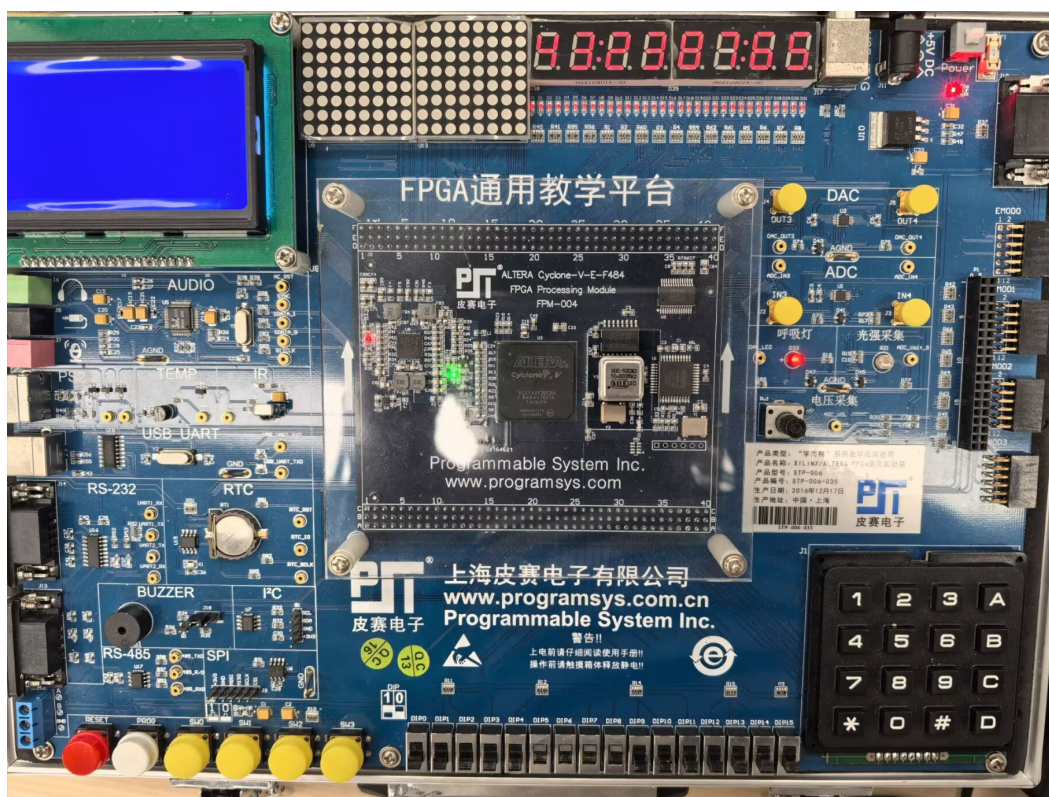
我选择 DIP0-2 作为位选信号输入，DIP3-6 作为数据信号输入，DIP7 作为使能信号输入，DIP8 作为复位信号输入，FPGA 板载时钟作为 clk 信号输入。

代码烧写进去后，我通过拨动 DIP 开关来改变数码管的显示内容，验证结果与预期一致，数码管能够正确显示对应的数字。

初始复位结果如下图所示，数码管显示数字 1-8：



随后我更改数码管 1 显示内容为 3，结果如下图所示，数码管能够正确显示对应的数字：



随后更改其余数码管的数字，结果均符合预期，验证成功。

此外，每次改变后拉低 reset 信号，数码管均能正确复位显示 1-8，验证成功。

六、实验总结

本次实验相对于实验一来说难度有所提升，需要我们综合运用之前所学的知识来实现一个较为复杂的电路设计。

刚开始设计电路的时候，编译过后代码一直报错，后面我结合 ai 排查了一下，发现是 wire 类型的数据不能在 always 模块中进行赋值操作，修改为 reg 类型后问题解决。

后面在实验过程中，我发现赋值操作基本能正常运行，但是一使用复位功能，数码管就只会显示其中一位数字，并且是随机显示其中一个数码管，后面经过排查发现是我在分频模块中加入了复位信号的判断，导致我复位时分频模块的输出信号一直是同一个电平，无法正确输出分频后的时钟信号，修改后问题解决。

本次实验我收获颇多，不仅加深了我对 Verilog 编程的理解，还提升了我解决实际问题的能力，更让我意识到了模块分开书写的重要性。